

SOCIETE «WaveOn» SARL
Code TVA: 1770422L/A/M/000
Siège Social : Rue Jendouba Lot n°14 ZI Mghira II - Fouchena 2082
E-mail : contact@waveon.tn
Tél : 36 010 390
Fax : 36010391



Documentation WebService WaveOn IoT

Réalisé à



Date de Réalisation : Le 18/03/2026

Année Administrative 2026

Documentation de l'API WaveOn IoT

Vue d'ensemble

La plateforme WaveOn IoT expose une API REST destinée à la supervision et au pilotage d'infrastructures connectées. Elle s'adresse aussi bien aux intégrateurs souhaitant développer des interfaces de monitoring qu'aux systèmes automatisés ayant besoin d'accéder aux données de terrain en temps réel.

- **URL de base :** https://iot.waveon.tn/WS_WAVEON/
Rq : pour les MetaData (Network info, user info, Device info..etc)
- **URL de Base pour la base de donnée NoSQL :**
https://iot.waveon.tn/WS_WAVEON/proxy/:81

Toutes les requêtes sont de type POST et doivent inclure dans leur corps les trois paramètres d'authentification **id_client**, **id_user** et **token**.

Un timeout global de 30 secondes est appliqué à chaque requête ; au-delà, le serveur retourne une erreur 504.

Concernant le stockage des données, une optimisation est en place : si la valeur reçue est identique à la dernière valeur enregistrée, elle n'est pas stockée. Une nouvelle entrée n'est donc créée que lorsqu'une variation est détectée.

1. Authentification — /LoginClientService/

Avant tout appel à l'API, il est nécessaire d'obtenir un **token** d'authentification ainsi que les identifiants **id_client** et **id_user**. Ces trois paramètres sont exigés par tous les autres endpoints.

L'authentification s'effectue via le service /LoginClientService/ en transmettant l'adresse email et le mot de passe de l'utilisateur.

Post URL : https://iot.waveon.tn/WS_WAVEON/LoginClientService/

Request Body :

```
{  
  "email": "amal.waly@etudiant-isi.utm.tn",  
  "password": "123456",  
  "devicename": "a989b76f5d8c827c"  
}
```

Response:

```
{
  "idclient": 171,
  "clientname": "amal",
  "email": "amal.waly@etudiant-isi.utm.tn",
  "username": "guest",
  "password": "8Sw5BCV1DybTyxxHGVYXXg==amal",
  "country": null,
  "city": null,
  "active": 0,
  "phonenumber": null,
  "numberofphones": 11,
  "jasonpath": null,
  "regiscode": null,
  "lastedate": null,
  "resetpasswordtoken": null,
  "isgateway": 0,
  "passkey": null,
  "unicastaddress": "0001",
  "isadmin": 0,
  "token": " YDCY8stdZrJvKc9S9kew",
  "iduser": 4,
  "devicename": null,
  "unicastlowaddress": "0001",
  "unicasthighaddress": 0,
  "grouplowaddress": 0,
  "grouphighaddress": 0,
  "scenelowaddress": 0,
  "scenehighaddress": 0,
  "sequencenumber": 0,
  "ivindex": 0,
  "mqttClientCreated": null,
  "mqttClientModified": null,
  "setUserAdminToken": null,
  "setUserGatewayToken": null,
  "mqttIP": null,
  "mqttPORT": null,
  "databaseIP": "iot.waveon.tn/WS_WAVEON/proxy/",
  "databasePORT": "81",
  "databaseUserName": "",
  "databasePassKey": "",
  "subscriptionType": 0,
  "maxUserReached": 0
}
```

En cas de succès, le serveur retourne un objet contenant l'ensemble des informations de session. Les champs essentiels à retenir pour les appels suivants sont `idclient` (ex. 171), `iduser` (ex. 11) et `token` (ex. "WFYj3yZEezJ9ppeQCa1U"). Il est important de noter

qu'un nouveau token est généré à chaque connexion ; les tokens précédents sont donc invalidés.

La réponse contient également `databaseIP` et `databasePORT`, qui correspondent respectivement à `iot.waveon.tn/WS_WAVEON/proxy/` et `81`, confirmant l'URL de base à utiliser pour les appels suivants.

2. Gestion des réseaux — `/getNetworksInfoService/`

Un client peut posséder plusieurs réseaux associés à son compte (par exemple : bâtiment 1, bâtiment 2, maison, etc.).

Chaque réseau correspond à une entité physique et regroupe un ensemble d'équipements liés. Les équipements d'un réseau peuvent être organisés en pièces (Rooms), permettant une structuration logique et hiérarchique.

Pour récupérer la liste des réseaux accessibles, on appelle ce service en fournissant les identifiants d'authentification ainsi que des timestamps de dernière mise à jour, qui permettent au serveur de ne retourner que les données modifiées depuis la dernière synchronisation. On peut utiliser "2019-10-01 01:01:01" comme valeur de référence.

Post URL :

`https://iot.waveon.tn/WS_WAVEON/getNetworksInfoService/`

Request Body :

```
{
  "idclient": 171,
  "iduser": 4,
  "token": "WfYj3yZEezJ9ppeQCa1U",
  "idNetwork": 2,
  "lastupdate": "2019-10-01 01:01:01",
  "commandLastId": 0,
  "permissionsLastUpdate": "2019-10-01 01:01:01",
  "roomsLastUpdate": "2019-10-01 01:01:01",
  "automationLastUpdate": "2019-10-01 01:01:01"
}
```

Response:

```
[
  {
    "idclient": 171,
    "name": "Network1",
    "directoryPath": "CL171REP/network1",
    "automationLastUpdate": "2026-02-12 11:01:58",
    "idNetwork": 1,
    "lastUpdate": "2025-09-12 15:11:27",
    "permissionsLastUpdate": "2026-02-10 20:57:54",
    "roomsLastUpdate": "2025-11-12 16:18:02",
    "securityLastUpdate": "2025-07-03 08:38:31",
    "commandLastId": 0
  }
]
```

```
},  
{  
  "idclient": 171,  
  "name": "Network2",  
  "directoryPath": "CL171REP/network2",  
  "automationLastUpdate": "2025-04-22 12:31:51",  
  "idNetwork": 2,  
  "lastUpdate": "2021-10-01 01:01:11",  
  "permissionsLastUpdate": "2025-06-24 14:37:34",  
  "roomsLastUpdate": "2025-11-05 13:03:01",  
  "securityLastUpdate": "2025-01-17 15:23:25",  
  "commandLastId": 0  
}  
]
```

La réponse est un tableau d'objets, chacun représentant un réseau avec ses métadonnées : `idNetwork` (identifiant unique du réseau), `name` (nom du réseau), `directoryPath` (chemin du répertoire associé côté serveur), ainsi que plusieurs timestamps indiquant la dernière date de modification des permissions, des pièces, des automatisations et de la sécurité. L'`idNetwork` récupéré ici est nécessaire pour tous les appels ultérieurs liés aux équipements de ce réseau.

récupère les données des capteurs
contrôler les équipements (allumer lum , ouvrir fen ,
régler clim..)

3. SmartControllers — /ClientGetAllSmartControllerService/

Les SmartControllers sont des équipements physiques directement connectés au serveur via IP. Ils sont identifiés par un `idSmartController`, un unicast (adresse unique de communication), et un `sn` (numéro de série).

La récupération se fait en précisant le réseau ciblé `idNetwork` ainsi que les timestamps de synchronisation :

Post URL :

`https://iot.waveon.tn/WS_WAVEON/ClientGetAllSmartControllerService/`

Request Body :

```
{  
  "idclient": 171,  
  "iduser": 4,  
  "idNetwork": 1,  
  "token": "WFYj3yZEezJ9ppeQCa1U",  
  "lastUpdate": "2019-10-01 01:01:01",  
  "commandLastId": 0,  
  "permissionsLastUpdate": "2019-10-01 01:01:01",  
  "roomsLastUpdate": "2019-10-01 01:01:01",  
  "automationLastUpdate": "2019-10-01 01:01:01"  
}
```

```
}
```

Response:

```
{
  "idclient": 171,
  "iduser": 4,
  "idnetwork": 1,
  "token": "",
  "controllers": [
    {
      "idSmartController": 1,
      "idNetwork": 1,
      "token":
"41mZKX97ScXQp1nSjIs2RNTruJ4seAwiJDbCPji2xYO3ELFK6p",
      "state": null,
      "elements": 1,
      "unicast": 66000,
      "name": "Compteur d'usine",
      "enabled": 1,
      "sn": "123456789_",
      "productKey": "productKey123456789",
      "hardwareVersion": "1",
      "firmwareVersion": "1",
      "productName": "SmartMeter",
      "productId": 12,
      "fabDate": "2024-12-27T17:56:28.632",
      "updateTargetFirmwareVersion": null,
      "newestFirmwareVersion": null,
      "newestFirmwareVersionLink": null,
      "lastStableFirmwareVersion": null,
      "lastStableFirmwareVersionLink": null,
      "lastModification": "2025-02-18T15:22:46.404",
      "wifiSSID": null,
      "wifiPasskey": null,
      "lastPing": null,
      "meterConfig":
"{\"power\":{\"unit\":\"KW\",\"scale\":10000,\"offset\":0},\"a
pparent_power\":{\"unit\":\"KVA\",\"scale\":10000,\"offset\":0
},\"reactive_power\":{\"unit\":\"KVAR\",\"scale\":10000,\"offs
et\":0},\"voltage\":{\"unit\":\"V\",\"scale\":10000,\"offset\"
:0},\"current\":{\"unit\":\"A\",\"scale\":10000,\"offset\":0},
\"frequency\":{\"unit\":\"Hz\",\"scale\":100,\"offset\":0},\"p
ower_factor\":{\"unit\":\"PF\",\"scale\":1000,\"offset\":0},\"
energy_active\":{\"unit\":\"Wh\",\"scale\":1,\"offset\":0},\"e
nergy_reactive\":{\"unit\":\"VARh\",\"scale\":1,\"offset\":0},
\"energy_apparent\":{\"unit\":\"VAh\",\"scale\":1,\"offset\":0
},\"var\":[\"tae\",\"tre\",\"tpf\"],\"max_power_capacity\":69}
",
    }
  ]
}
```

```

        "modelName": null,
        "ac_Protocol": null,
        "tv_Protocol": null,
        "stereo_Protocol": null,
        "fan_Protocol": null,
        "radio_Protocol": null,
        "rgb_Protocol": null,
        "light_Protocol": null,
        "box_Protocol": null
    },
    {
        "idSmartController": 2,
        "idNetwork": 1,
        "token":
"UYcbsMUY7WzKTBFhFOhwMOS4BgmqFQoasQVSf51Ss9UvrLVp5J",
        "state": null,
        "elements": 1,
        "unicast": 66001,
        "name": "Compteur Test",
        "enabled": 1,
        "sn": "123456789_2",
        "productKey": "productKey123456789_2",
        "hardwareVersion": "1",
        "firmwareVersion": "1",
        "productName": "SmartMeter",
        "productId": 12,
        "fabDate": "2024-12-27T17:56:28.632",
        "updateTargetFirmwareVersion": null,
        "newestFirmwareVersion": null,
        "newestFirmwareVersionLink": null,
        "lastStableFirmwareVersion": null,
        "lastStableFirmwareVersionLink": null,
        "lastModification": "2025-02-18T15:26:37.601",
        "wifiSSID": null,
        "wifiPasskey": null,
        "lastPing": null,
        "meterConfig":
{"\power\":{"unit\":"KW",\scale\:1000,\offset\:0},\ap
parent_power\":{"unit\":"KVA",\scale\:1000,\offset\:0},
\reactive_power\":{"unit\":"KVAR",\scale\:1000,\offset\
":0},\voltage\":{"unit\":"V",\scale\:100,\offset\:0},\
current\":{"unit\":"A",\scale\:100,\offset\:0},\frequ
ency\":{"unit\":"Hz",\scale\:100,\offset\:0},\power_fa
ctor\":{"unit\":"PF",\scale\:1000,\offset\:0},\energy_
active\":{"unit\":"KWh",\scale\:0.01,\offset\:0},\ener
gy_reactive\":{"unit\":"VARh",\scale\:1000,\offset\:0},
\energy_apparent\":{"unit\":"VAh",\scale\:1,\offset\:0
},\var\":[\tpf",\aed",\aed+r",\aed-
r",\red",\aer"],\max_power_capacity\:null}],
        "modelName": null,
        "ac_Protocol": null,

```

```
        "tv_Protocol": null,
        "stereo_Protocol": null,
        "fan_Protocol": null,
        "radio_Protocol": null,
        "rgb_Protocol": null,
        "light_Protocol": null,
        "box_Protocol": null
    },
    {
        "idSmartController": 3,
        "idNetwork": 1,
        "token": null,
        "state": null,
        "elements": 1,
        "unicast": 66300,
        "name": "Debit Metre General",
        "enabled": 1,
        "sn": "Gprs_Flow_Meter_001",
        "productKey": "Gprs_Flow_Meter_001",
        "hardwareVersion": "v1.0",
        "firmwareVersion": "v1.0",
        "productName": "Gprs Flow Meter",
        "productId": 13,
        "fabDate": null,
        "updateTargetFirmwareVersion": null,
        "newestFirmwareVersion": null,
        "newestFirmwareVersionLink": null,
        "lastStableFirmwareVersion": null,
        "lastStableFirmwareVersionLink": null,
        "lastModification": null,
        "wifiSSID": null,
        "wifiPasskey": null,
        "lastPing": null,
        "meterConfig":
        "{\\"flow_rate\\":{\\"unit\\":\\"L/min\\",\\"scale\\":2.22222223,\\"offset\\":0},\\"total_flow\\":{\\"unit\\":\\"L\\",\\"scale\\":27,\\"offset\\":0},\\"var\\":[\\"flow_rate\\",\\"total_flow\\"]}",
        "modelName": "WaveOn_MC60_Flow_Meter",
        "ac_Protocol": null,
        "tv_Protocol": null,
        "stereo_Protocol": null,
        "fan_Protocol": null,
        "radio_Protocol": null,
        "rgb_Protocol": null,
        "light_Protocol": null,
        "box_Protocol": null
    }
],
    "controllerLastUpdate": ""
}
```


Chaque Smart Controller est identifié de manière unique par trois paramètres : `unicast` (adresse unique dans le réseau mesh, ex : 66000, 66001), **Serial Number (sn)** (numéro de série physique de l'équipement) et `idSmartController` (identifiant numérique dans la base de données).

Parmi les champs aussi : `productName` et `productId` indiquent le type de l'appareil ; `name` est le nom lisible attribué par l'utilisateur ; `enabled` indique si l'appareil est actif.

On a deux types de SmartControllers :

Le **SmartMeter** (`productId: 12`) est un compteur électrique qui mesure des grandeurs telles que la puissance, la tension, le courant, le facteur de puissance et les énergies actives, réactives et apparentes. Le **Gprs Flow Meter** (`productId: 13`) est un débitmètre qui mesure le débit instantané (`flow_rate` en L/min) et le débit total cumulé (`total_flow` en litres).

Le Champ `meterConfig` : Ce champ JSON embarqué décrit la configuration de mesure propre à chaque SmartController. Il précise, pour chaque grandeur physique mesurée, l'unité (`unit`), le facteur d'échelle (`scale`) et le **décalage** (`offset`) à appliquer pour convertir la valeur brute reçue en valeur réelle.

Les grandeurs standards comme la puissance, la tension, le courant ou la fréquence sont disponibles par défaut.

Il contient un tableau `var` qui liste spécifiquement les variables d'énergie disponibles sur cet équipement (par exemple : `tae`, `tre`, `tpf`). De plus, si le compteur dispose d'une puissance maximale, celle-ci est indiquée via le champ `max_power_capacity` dans le `meterConfig`.

Structure type pour un SmartMeter :

```
{
  "power": {"unit": "KW", "scale": 10000, "offset": 0},
  "voltage": {"unit": "V", "scale": 10000, "offset": 0},
  "current": {"unit": "A", "scale": 10000, "offset": 0},
  "frequency": {"unit": "Hz", "scale": 100, "offset": 0},
  "energy_active": {"unit": "Wh", "scale": 1, "offset": 0},
  "var": ["tae", "tre", "tpf"],
  "max_power_capacity": 69
}
```

Structure type pour un débitmètre :

```
{
  "flow_rate": {"unit": "L/min", "scale": 2,2, "offset": 0},
  "total_flow": {"unit": "L", "scale": 27, "offset": 0},
  "var": ["flow_rate", "total_flow"]
}
```

La formule de conversion est : $\text{valeur_réelle} = (\text{valeur_brute} / \text{scale}) + \text{offset}$

4. Équipements Bluetooth Mesh — /downloadFile/

Les équipements Bluetooth Mesh (interrupteurs, capteurs, variateurs, volets, etc.) d'un réseau `idNetwork` ne sont pas listés directement via une API dédiée, mais sont décrits dans un fichier de configuration réseau au format JSON, téléchargeable via ce service. **Post URL :**
`https://iot.waveon.tn/WS_WAVEON/downloadMultiNetworkFile/`

Request Body :

```
{
  "idclient": 171,
  "iduser": 4,
  "token": "WFYj3yZEezJ9ppeQCa1U",
  "lastupdate": "2019-10-01 01:01:01",
  "idNetwork": 1,
  "permissionsLastUpdate": "2019-10-01 01:01:01",
  "roomsLastUpdate": "2019-10-01 01:01:01",
  "automationLastUpdate": "2019-10-01 01:01:01"
}
```

Le fichier retourné est un fichier de configuration Bluetooth Mesh conforme au standard BLE Mesh. Il contient les clés réseau (`netKeys`), les clés applicatives (`appKeys`), la liste des provisioners (utilisateurs gestionnaires du réseau), ainsi que la liste complète des nœuds (`nodes`) et des groupes (`groups`).

Classification des équipements par pid

Chaque nœud est identifié par deux champs : `cid` (Company Identifier, toujours 0059 pour WaveOn) et `pid` (Product Identifier), qui détermine le type de l'équipement. Les PID disponible sont:

`pid: 0000` — équipements génériques simples (ex. ventilateur, bureau RDC) `pid: 0001` — WaveOn Remote (télécommande)

`pid: 0002` — équipements double canal, le type le plus courant (interrupteurs, capteurs, dimmers)

`pid: 0003` — commandes de volets roulants (blinds)

`pid: 0004` — WaveOn Booster (amplificateur de signal)

`pid: 0005` — équipements à deux canaux de type luminaire (double lampe) `pid: 0007` — climatiseur

`pid: 0008` — actionneur de porte

`pid: 0009` — capteur de porte

Chaque nœud est identifiable via son `unicastAddress` (adresse unique dans le réseau mesh), son `UUID` (identifiant global), et son `name` (nom lisible). Il peut contenir plusieurs `elements`, chacun représentant un canal indépendant de l'appareil, avec ses propres `models` de communication mesh.

1. Node

Un **Node** représente un équipement complet dans le réseau.
Il possède une **adresse principale** (`unicastAddress`) et contient un ou plusieurs **Elements**.

Exemple :

```
{
  "name": "Salle Serveur",
  "unicastAddress": "001F",
  "elements": [...]
}
```

Caractéristiques principales :

- Un **Node** correspond à un appareil physique unique.
- Il possède une **adresse principale**.
- Il regroupe plusieurs **Elements**.
- Chaque Node peut exposer plusieurs fonctionnalités via ses Elements et Models.

2. Element

Un **Element** est une partie logique du Node.

Chaque **Element** possède sa propre **adresse indépendante** dérivée de l'adresse principale du Node.

Exemple :

```
{
  "name": "Element: 0x001F",
  "index": 0,
  "models": [...]
}
```

Caractéristiques principales :

- Un Node peut contenir plusieurs Elements.
- Chaque Element correspond à une **adresse indépendante**.
- Un Element regroupe un ensemble de **Models**.
- Chaque Element peut représenter une partie fonctionnelle différente de l'appareil.

Dans l'exemple :

SOCIETE «WaveOn» SARL

Siège Social : Rue Jendouba Lot n°14 ZI Mghira II - Fouchena 2082 Code TVA: 1770422L/A/M/000

E-mail : contact@waveon.tn Site Web : www.waveon.tn

Tél : 36 010 390

Fax : 36010391

- Element 0 → adresse 0x001F correspond à l'adresse de la première lampe
- Element 1 → adresse 0x0020 correspond à l'adresse de la Deuxième

3. Model

Un **Model** représente une **fonctionnalité spécifique** d'un Element.

Chaque Model est identifié par un **modelId**.

Exemple :

```
{  
  "modelId": "1002",  
  "bind": [0],  
  "subscribe": ["C000"]  
}
```

Caractéristiques principales :

- Chaque Model correspond à une **fonctionnalité précise**.
- Un Element peut contenir plusieurs Models.
- Chaque Model peut :
 - être associé à une clé (**bind**)
 - recevoir des messages (**subscribe**)
 - envoyer des messages (**publish**)

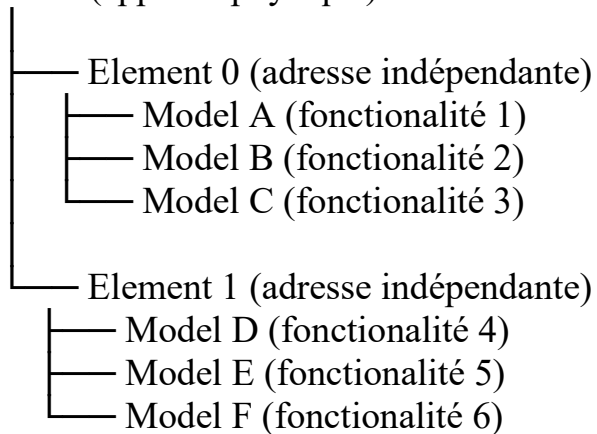
ModelIDs:

GENERIC_LEVEL_SERVER = 0x1002; (Controle des equipments)
HUMIDITY_SERVER = 0x00590011;
TEMPERTURE_SERVER = 0x00590012
OCCUPANCY_SERVER = 0x00590013;
LUMINOSITY_SERVER = 0x00590014
COUNTER_SERVER = 0x00590015;
BLINDS_SERVER = 0x00590016;
METERING_SERVER = 0x00590018; (For power Metering and Others)

4. Relation entre Node, Element et Model

Structure hiérarchique :

Node (appareil physique)



5. Résumé

- **Node** = appareil complet
- **Element** = adresse indépendante à l'intérieur du Node
- **Model** = fonctionnalité spécifique de l'Element

5. Paramètres électriques disponibles

Les endpoints de données énergétiques utilisent des abréviations pour identifier les grandeurs électriques. Le paramètre accepte indifféremment le nom complet ou l'abréviation.

Lorsqu'on utilise le nom complet, les espaces doivent être encodés en URL, par exemple : total%20reactive%20power.

Catégorie	Nom complet	Abréviation
Énergie active totale	total active energy	tae
Énergie réactive totale	total reactive energy	tre
Énergie active livrée	active energy delivered	aed
Énergie active reçue	active energy received	aer
Énergie active livrée + reçue	active energy delivered + received	aed+r
Énergie active livrée – reçue	active energy delivered - received	aed-r
Énergie réactive livrée	reactive energy delivered	red
Énergie réactive reçue	reactive energy received	rer
Énergie réactive livrée + reçue	reactive energy delivered + received	red+r
Énergie réactive livrée – reçue	reactive energy delivered - received	red-r
Énergie apparente livrée	apparent energy delivered	aped
Énergie apparente reçue	apparent energy received	aper
Énergie apparente livrée + reçue	apparent energy delivered + received	aped+r
Énergie apparente livrée – reçue	apparent energy delivered - received	aped-r
Tensions	a/b/c phase voltage	va, vb, vc
Courants	a/b/c/n/g phase current	ia, ib, ic, in, ig
Puissances actives	total active power / a/b/c phase	tap, pa, pb, pc
Puissances réactives	total reactive power / a/b/c phase	trp, rpa, rpb, rpc
Puissance apparente	total apparent power	tapp
Facteur de puissance	total / a/b/c phase power factor	tpf, pfa, pfb, pfc
Fréquence	frequency	freq
Énergie par phase	activeEnergyA/B/C, reactiveEnergyA/B/C	aea, aeb, aec, rea, reb, rec

6. Données énergétiques

tae

6.1 Historique brut — /GetClientGenericEnergyValue/{parameter}

Description : Récupère l'historique brut (minute par minute) d'un paramètre électrique pour un compteur donné sur une plage de temps.

Méthode : POST

URL :

https://iot.waveon.tn/WS_WAVEON/proxy/:81/GetClientGenericEnergyValue/{parameter}

Exemple de Request Body :

```
{ "unicast": 66000,
  "id_client": 171,
  "id_network": 1,
  "id_user": 4,
  "token": "WFYj3yZEezJ9ppeQCa1U",
  "start_time": "2026-01-01T11:59",
  "end_time": "2026-01-01T12:15" }
```

Response (200):

```
{"unicast":66000,"parameter":"tae","start_time":"2026-01-01T11:59","end_time":"2026-01-01T12:15",
"history":[{"timestamp":"2026-01-01T11:59:00","value":1702593},
{"timestamp":"2026-01-01T12:00:00","value":1702594},{"timestamp":"2026-01-01T12:01:00","value":1702596},
{"timestamp":"2026-01-01T12:02:00","value":1702598},{"timestamp":"2026-01-01T12:04:00","value":1702599},
{"timestamp":"2026-01-01T12:05:00","value":1702601},{"timestamp":"2026-01-01T12:06:00","value":1702602},
{"timestamp":"2026-01-01T12:08:00","value":1702604},{"timestamp":"2026-01-01T12:09:00","value":1702605},
{"timestamp":"2026-01-01T12:10:00","value":1702607},{"timestamp":"2026-01-01T12:12:00","value":1702608},
{"timestamp":"2026-01-01T12:13:00","value":1702610},{"timestamp":"2026-01-01T12:14:00","value":1702611},
{"timestamp":"2026-01-01T12:15:00","value":1702613}]}
```

Le paramètre est passé directement dans l'URL,

par exemple POST /GetClientGenericEnergyValue/tae. La réponse retourne un tableau `history` où chaque entrée contient un `timestamp` et la valeur brute correspondante. Grâce à l'optimisation de stockage, seules les minutes où la valeur a changé sont présentes dans l'historique.

Les codes d'erreur possibles sont 400 (champs obligatoires manquants), 403 (authentification échouée), 404 (paramètre non reconnu dans le mapping) et 500 (erreur serveur interne).

6.2 Moyenne journalière — /GetClientDailyAverage/{parameter}

Description : Récupère l'historique agrégé par jour d'un paramètre. Pour les paramètres cumulatifs (énergie totale active/réactive : tae, tre), le calcul est (dernière valeur – première valeur) du jour. Pour les autres paramètres, c'est la **moyenne** journalière.

Méthode : POST

URL : /GetClientDailyAverage/{parameter}

Exemple de Request Body:

```
{ "unicast": 66000,
  "id_client": 171,
  "id_network": 1,
  "id_user": 11,
  "token": "WFYj3yZEezJ9ppeQCa1U",
  "start_time": "2026-02-01T00:00",
  "end_time": "2026-02-06T23:59"
}
```

Response (200):

```
{ "unicast": 66000, "parameter": "tae", "start_time": "2026-02-01T00:00", "end_time": "2026-02-06T23:59",
  "daily_history": [ { "timestamp": "2026-02-01T00:00:00", "value": 1769.0 },
    { "timestamp": "2026-02-02T00:00:00", "value": 6142.0 },
    { "timestamp": "2026-02-03T00:00:00", "value": 5952.0 },
    { "timestamp": "2026-02-04T00:00:00", "value": 6196.0 },
    { "timestamp": "2026-02-05T00:00:00", "value": 6244.0 },
    { "timestamp": "2026-02-06T00:00:00", "value": 4425.0 } ] }
```

Logique de calcul :

Type de paramètre	Calcul	Exemples
Cumulatif	last_value – first_value du jour	Les paramètres énergétiques : tae, tre, etc
Non cumulatif	Moyenne des valeurs du jour	Les autres paramètres : ia, freq, etc.

Utilisation côté front-end :

Granularité	Endpoint appelé	Agrégation client
Daily (histogramme)	GetClientGenericEnergyValue	Grouper par jour (last – first)
Monthly	GetClientDailyAverage	Somme des jours par mois
Yearly	GetClientDailyAverage	Somme des jours par année

7. Données environnementales

Ces quatre services partagent la même structure de requête : `unicast`, `id_client`, `id_network`, `id_user`, `token`, `start_timestamp` et `end_timestamp`. Seule la clé de retour dans la réponse change selon la grandeur mesurée.

7.1 Température — /getTempHistory/

Description : Récupère l'historique de température sur une plage de temps.

Temp = val/100

Méthode: POST

URL: /getTempHistory/

Request Body:

```
{
  "unicast": 3,
  "id_client": 171,
  "id_network": 1,
  "id_user": 4,
  "token": "f6iOQUEEBNHNhz6jq5XC",
  "start_timestamp": "2026-03-01T09:00:00",
  "end_timestamp": "2026-03-01T09:10:00"
}
```

Response (200):

```
{
  "unicast": 3,
  "id_client": 171,
  "id_network": 1,
  "id_user": 11,
  "start_timestamp": "2026-03-01T09:00:00",
  "end_timestamp": "2026-03-01T09:10:00",
  "temp": 100
}
```

```
"temperature_history":[{"timestamp":"2026-03-01T09:02:00","temperature":2100},
{"timestamp":"2026-03-01T09:03:00","temperature":2101},
{"timestamp":"2026-03-01T09:05:00","temperature":2103},
{"timestamp":"2026-03-01T09:06:00","temperature":2105},
{"timestamp":"2026-03-01T09:07:00","temperature":2103},
{"timestamp":"2026-03-01T09:09:00","temperature":2102},
{"timestamp":"2026-03-01T09:10:00","temperature":2106}]}
```

7.2 Humidité — /getHumHistory/

Description: Récupère l'historique d'humidité. Humidité relative = valeur /100

Méthode: POST

URL: /getHumHistory/

Request Body :

```
{"unicast":3,
"id_client":171,
"id_network": 1,
"id_user": 4,
"token": "f6iOQUEEBNHNhz6jq5XC",
"start_timestamp": "2026-03-01T09:00:00",
"end_timestamp": "2026-03-01T09:10:00"}
```

Response (200):

```
{"unicast":3,"id_client":171,"id_network":1,"id_user":11,"start_timestamp":"2026-03-01T09:00:00","end_timestamp":"2026-03-01T09:10:00",
"humidity_history":[{"timestamp":"2026-03-01T09:00:00","humidity":6194},
{"timestamp":"2026-03-01T09:03:00","humidity":6192},
{"timestamp":"2026-03-01T09:05:00","humidity":6195},
{"timestamp":"2026-03-01T09:06:00","humidity":6194},
{"timestamp":"2026-03-01T09:07:00","humidity":6192},
```

7.3 Luminosité — /getLumHistory/

Description : Récupère l'historique de luminosité. Luminosité absolue en %

Méthode : POST

URL: /getLumHistory/

Request Body:

```
{"unicast":2,  
  "id_client":171,  
  "id_network": 1,  
  "id_user": 4,  
  "token": "f6iOQUEEBNHNhz6jq5XC",  
  "start_timestamp": "2026-03-01T09:00:00",  
  "end_timestamp": "2026-03-01T09:30:00"}
```

Response (200):

```
{"unicast":2,"id_client":171,"id_network":1,"id_user":11,"start_timestamp":"2026-03-01T09:00:00","end_timestamp":"2026-03-01T09:30:00",  
  "luminosity_history":[{"timestamp":"2026-03-01T09:09:00","luminosity":18},  
    {"timestamp":"2026-03-01T09:16:00","luminosity":19},  
    {"timestamp":"2026-03-01T09:17:00","luminosity":23},  
    {"timestamp":"2026-03-01T09:18:00","luminosity":24},  
    {"timestamp":"2026-03-01T09:20:00","luminosity":22},  
    {"timestamp":"2026-03-01T09:21:00","luminosity":20},  
    {"timestamp":"2026-03-01T09:22:00","luminosity":19}]}
```

7.4 Niveau de lumière — /getLightLevelHistory/

Description : Récupère l'historique du niveau de lumière (light level) : Les valeurs sont binaires : 32767 indique une présence de lumière (ON) et -32768 indique l'absence de lumière (OFF).

Méthode: POST

URL: /getLightLevelHistory/

Request Body:

```
{"unicast":3,  
  "id_client":171,  
  "id_network": 1,  
  "id_user": 4,  
  "token": "f6iOQUEEBNHNhz6jq5XC",
```

```
"start_timestamp": "2026-03-01T09:00:00",  
"end_timestamp": "2026-03-04T09:30:00"  
}
```

Response (200):

```
{"unicast":3,"id_client":171,"id_network":1,"id_user":11,"start_timestamp":"2026-03-01T09:00:00","end_timestamp":"2026-03-04T09:30:00",  
"lightlevel_history":[{"timestamp":"2026-03-02T07:26:00","lightlevel":32767},  
{"timestamp":"2026-03-02T10:49:00","lightlevel":-32768},  
{"timestamp":"2026-03-02T13:04:00","lightlevel":32767},  
{"timestamp":"2026-03-02T16:11:00","lightlevel":-32768},  
{"timestamp":"2026-03-03T06:53:00","lightlevel":32767},  
{"timestamp":"2026-03-03T14:18:00","lightlevel":-32768}]}
```

7.5 Occupation — /getOccupancyHistory/

Description : Récupère l'historique d'occupation (présence). La valeur 1 signifie qu'une présence a été détectée, 0 indique une absence. Le champ peut avoir d'autre valeur personnalisé selon le produit .

Méthode : POST

URL : /getOccupancyHistory/

Request Body :

```
{"unicast":3,  
"id_client":171,  
"id_network": 1,  
"id_user": 4,  
"token": "f6iOQUEEBNHNhz6jq5XC",  
"start_timestamp": "2026-03-02T09:00:00",  
"end_timestamp": "2026-03-02T10:30:00"  
}
```

Response (200):

```
{"unicast":3,"id_client":171,"id_network":1,"id_user":11,"start_timestamp":"2026-03-02T09:00:00","end_timestamp":"2026-03-02T10:30:00",  
"occupancy_history":[{"timestamp":"2026-03-02T10:02:00","occupancy":1},
```

```
{"timestamp":"2026-03-02T10:03:00","occupancy":1},  
{"timestamp":"2026-03-02T10:04:00","occupancy":0},  
{"timestamp":"2026-03-02T10:05:00","occupancy":1},  
{"timestamp":"2026-03-02T10:07:00","occupancy":0}}}
```

7.6 Niveau de comptage — /getMeteringLevelHistory/

consommation d'un clim
,lampe ,salle ..

Description : Récupère l'historique du niveau de comptage (metering level).

Méthode: POST

URL: /getMeteringLevelHistory/

Request Body:

```
{"unicast":14,  
"id_client":171,  
"id_network": 1,  
"id_user": 4,  
"token": "f6iOQUEEBNHNhz6jq5XC",  
"start_timestamp": "2026-03-02T09:00:00",  
"end_timestamp": "2026-03-02T09:30:00"}
```

Response (200):

```
{"unicast":14,"id_client":171,"id_network":1,"id_user":11,"start_timestamp":"2026-03-02T09:00:00","end_timestamp":"2026-03-02T09:30:00",  
"metering_history":[{"timestamp":"2026-03-02T09:09:00","meteringlevel":10.0},  
{"timestamp":"2026-03-02T09:10:00","meteringlevel":20.0},  
{"timestamp":"2026-03-02T09:19:00","meteringlevel":10.0},  
{"timestamp":"2026-03-02T09:20:00","meteringlevel":20.0},  
{"timestamp":"2026-03-02T09:29:00","meteringlevel":10.0},  
{"timestamp":"2026-03-02T09:30:00","meteringlevel":20.0}]}}
```